Name: v.preetham reddy
Roll no:2303A52372
Batch No: 40

Task Description #1

(Password Strength Validator – Apply AI in
Security Context)
• Task: Apply AI to generate at least 3 assert test cases for
is_strong_password(password) and implement the validator
function.
• Requirements:
o Password must have at least 8 characters.
o Must include uppercase, lowercase, digit, and special
character.
o Must not contain spaces.
Example Assert Test Cases:
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234") == True

Prompt:

```
#test-driven Development with AI – Generating and Working with Test Cases
#Task 1-Password strength validator -Apply AI in security content
#Password must have at least 8 characters.
# Must include uppercase, lowercase, digit, and special
#character. Must not contain spaces.
```

```
#change the password string and integers also to test different cases
#use proper if statement if the input string is wrong or right and print the result
accordingly
```

Code:

```
#test-driven Development with AI – Generating and Working with Test Cases
#Task 1-Password strength validator -Apply AI in security content
#Password must have at least 8 characters.
# Must include uppercase, lowercase, digit, and special
#character. Must not contain spaces.
def password(password_string):
    if len(password_string) < 8:
        return False
    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False
    special_characters = "!@#$%^&*()-+"
    for char in password_string:
        if char.isupper():
```

```
            has_upper = True
        elif char.islower():
            has_lower = True
        elif char.isdigit():
            has_digit = True
        elif char in special_characters:
            has_special = True
        elif char == " ":
            return False  # Contains space, invalid password

    return has_upper and has_lower and has_digit and has_special
#Test cases give in userdefined input
password_string = input("Enter a password to validate: ")
#change the password string and integers also to test different cases
#use proper if statement if the input string is wrong or right and print the result
accordingly
if password(password_string):
    print("Password is strong.")
else:
    print("Password is weak. It must  at least 8 characters long, include uppercase,
lowercase, digit, and special character, and must not contain spaces.")
```

Output:

```
C:\Users\dell\OneDrive\Documents>C:/Users/dell/AppData/Local/Programs/Python/Python314/python.exe c:/Users/dell/OneD
rive/Documents/AI_Ass_Lab_Ass_8.1.py
Enter a password to validate: Bharath@2005
Password is strong.

C:\Users\dell\OneDrive\Documents>C:/Users/dell/AppData/Local/Programs/Python/Python314/python.exe c:/Users/dell/OneD
rive/Documents/AI_Ass_Lab_Ass_8.1.py
Enter a password to validate: radha&230
Password is weak. It must  at least 8 characters long, include uppercase, lowercase, digit, and special character, a
nd must not contain spaces.
```

Task Description #2

(Number Classification with Loops – Apply
AI for Edge Case Handling)
• Task: Use AI to generate at least 3 assert test cases for a
classify_number(n) function. Implement using loops.

• Requirements:
o Classify numbers as Positive, Negative, or Zero.
o Handle invalid inputs like strings and None.
o Include boundary conditions (-1, 0, 1).
Example Assert Test Cases:
assert classify_number(10) == "Positive"

assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"

Prompt:

```
#Number classification with loops Apply Ai for edge case Handling
#Classify numbers as Positive, Negative, or Zero.
#Handle invalid inputs like strings and None.
#Include boundary conditions (-1, 0, 1).
#give a primary code for number classification and then test it with different cases
give Test cases of user defined for number classification
```

Code:

```
def classify_number(num):
    if num is None:
        return "Invalid input: None is not a number."
    if isinstance(num, str):
        return "Invalid input: String is not a number."
    if num > 0:
        return "Positive"
    elif num < 0:
        return "Negative"
    else:
        return "Zero"
# give Test cases of user defined for number classification
test_cases = [1, -1, 0, None, "abc"]
for case in test_cases:
    print(f"Input: {case}, Output: {classify_number(case)}")
```

Output:

```
c:\Users\dell\OneDrive\Documents>
c:\Users\dell\OneDrive\Documents> c: && cd c:\Users\dell\OneDrive\Documents && cmd /C "c:\Users\dell\AppData\Local\P
rograms\Python\Python314\python.exe c:\Users\dell\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\l
ibs\debugpy\launcher 52504 -- C:\Users\dell\OneDrive\Documents\AI_Ass_Lab_Ass_8.1.py "
Input: 1, Output: Positive
Input: -1, Output: Negative
Input: 0, Output: Zero
Input: None, Output: Invalid input: None is not a number.
Input: abc, Output: Invalid input: String is not a number.
```

Activate Windows

Task Description #3

(Anagram Checker – Apply AI for String Analysis)
• Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.
• Requirements:

o Ignore case, spaces, and punctuation.
o Handle edge cases (empty strings, identical words)

Prompt:

```
#anagram checker with AI for string analysis
#give code for but ignore case,spaces,and punctuation and handle edge cases like
empty strings and non-string inputs
# user defined Test cases for anagram checker give some false statements also to
check the validity of the code
```

Code:

```
import string
def clean_string(s):
    return ''.join(char.lower() for char in s if char.isalnum())
def are_anagrams(str1, str2):
    if not isinstance(str1, str) or not isinstance(str2, str):
        return "Invalid input: Both inputs must be strings."
    cleaned_str1 = clean_string(str1)
    cleaned_str2 = clean_string(str2)
    return sorted(cleaned_str1) == sorted(cleaned_str2)
# user defined Test cases for anagram checker give some false statements also to
check the validity of the code
test_cases = [
    ("listen", "silent"),  # True
    ("Triangle", "Integral"),  # True
    ("Apple", "Pabble"),  # False
    ("Dormitory", "Dirty Room"),  # True
    ("Conversation", "Voices Rant On"),  # True
    ("Astronomer", "Moon man"),  # True
    ("The eyes", "They see"),  # True
    ("A gentleman", "Elegant man"),  # True
    ("Clint Eastwood", "Old West Action"),  # True
    ("Software", "Swear oft"),  # False
    ("", ""),  # True (both are empty)
    (None, "test"),  # Invalid input
    ("test", None),  # Invalid input
    (123, "321"),  # Invalid input
    ("123", 321),  # Invalid input
]
for str1, str2 in test_cases:
    result = are_anagrams(str1, str2)
    print(f"Input: '{str1}' and '{str2}', Output: {result}")
```

Output:

```
rograms\Python\Python314\python.exe c:\Users\dell\.vscode\extensions\ms-python.debugpy-2025.18.0
ibs\debugpy\launcher 57480 -- C:\Users\dell\OneDrive\Documents\AI_Ass_Lab_Ass_8.1.py "
Input: 'listen' and 'silent', Output: True
Input: 'Triangle' and 'Integral', Output: True
Input: 'Apple' and 'Pabble', Output: False
Input: 'Dormitory' and 'Dirty Room', Output: True
Input: 'Conversation' and 'Voices Rant On', Output: True
Input: 'Astronomer' and 'Moon man', Output: False
Input: 'The eyes' and 'They see', Output: True
Input: 'A gentleman' and 'Elegant man', Output: True
Input: 'Clint Eastwood' and 'Old West Action', Output: True
Input: 'Software' and 'Swear oft', Output: True
Input: '' and '', Output: True
Input: 'None' and 'test', Output: Invalid input: Both inputs must be strings.
Input: 'test' and 'None', Output: Invalid input: Both inputs must be strings.
Input: '123' and '321', Output: Invalid input: Both inputs must be strings.
Input: '123' and '321', Output: Invalid input: Both inputs must be strings.
```

Task Description #4

(Inventory Class – Apply AI to Simulate Real-World Inventory System)
• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
• Methods:
o add_item(name, quantity)
o remove_item(name, quantity)

Prompt:

```
#Inventory class- Apply Ai to simulate real-world Inventory System
#add_item(name, quantity),remove_item(name, quantity) get_stock(name)
# user defined test cases for inventory class
```

```
class Inventory:
    def __init__(self):
        self.stock = {}
    def add_item(self, name, quantity):
        if name in self.stock:
            self.stock[name] += quantity
        else:
            self.stock[name] = quantity
    def remove_item(self, name, quantity):
        if name in self.stock and self.stock[name] >= quantity:
            self.stock[name] -= quantity
            return True
        return False
```

```
    def get_stock(self, name):
        return self.stock.get(name, 0)
# user defined test cases for inventory class
inventory = Inventory()
inventory.add_item("apple", 10)
inventory.add_item("banana", 20)
print(inventory.get_stock("apple"))  # Output: 10
print(inventory.get_stock("banana"))  # Output: 20
print(inventory.get_stock("orange"))  # Output: 0 (not in stock)
print(inventory.remove_item("apple", 5))  # Output: True
print(inventory.get_stock("apple"))  # Output: 5
print(inventory.remove_item("banana", 25))  # Output: False (not enough stock)
print(inventory.get_stock("banana"))  # Output: 20 (unchanged)
inventory.add_item("apple", 5)
print(inventory.get_stock("apple"))  # Output: 10 (5 added to existing stock)
```

Output:

```
rograms\Python\Python314\python.exe c:\Users\dell\.vscode\extensions\ms-python.debugpy-202
ibs\debugpy\launcher 61340 -- C:\Users\dell\OneDrive\Documents\AI_Ass_Lab_Ass_8.1.py "
10
20
0
True
5
False
20
10
```

Task Description #5

(Date Validation & Formatting – Apply AI for
Data Validation)
• Task: Use AI to generate at least 3 assert test cases for
validate_and_format_date(date_str) to check and convert
dates.
• Requirements:
o Validate "MM/DD/YYYY" format.
o Handle invalid dates.
o Convert valid dates to "YYYY-MM-DD".

Prompt:

```
#Date validation formatting with AI for date Validation
#validate date format (DD/MM/YYYY) and handle edge cases like leap years and
invalid dates
#dont use comments
```

Code:

```python
import re
def is_leap_year(year):
    return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
def validate_date(date_string):
    pattern = r'^(0[1-9]|[12][0-9]|3[01])/(0[1-9]|1[0-2])/(19|20)\d{2}$'
    if not re.match(pattern, date_string):
        return "Invalid date format. Please use DD/MM/YYYY."
    day, month, year = map(int, date_string.split('/'))
    if month in [1, 3, 5, 7, 8, 10, 12]:
        return "Valid date." if day <= 31 else "Invalid date."
    elif month in [4, 6, 9, 11]:
        return "Valid date." if day <= 30 else "Invalid date."
    elif month == 2:
        if is_leap_year(year):
            return "Valid date." if day <= 29 else "Invalid date."
        else:
            return "Valid date." if day <= 28 else "Invalid date."
#give user defined test cases for date validation including valid and invalid dates, leap years, and edge cases
test_dates = [
    "29/02/2020",  # Valid leap year date
    "29/02/2019",  # Invalid non-leap year date
    "31/04/2021",  # Invalid date (April has 30 days)
    "30/06/2021",  # Valid date
    "31/12/2021",  # Valid date
    "01/01/2021",  # Valid date
    "31/11/2021",  # Invalid date (November has 30 days)
    "15/08/1947",  # Valid date
    "31/09/2021",  # Invalid date (September has 30 days)
    "00/01/2021",  # Invalid day
    "01/00/2021",  # Invalid month
    "32/01/2021",  # Invalid day
    "01/13/2021",  # Invalid month
    "abc",         # Invalid format
    "12-12-2021"   # Invalid format
]
for date in test_dates:
     result = validate_date(date)
         print(f"Input: '{date}', Output: {result}")
```

Output:

```
Input: '31/04/2021', Output: Invalid date.
Input: '30/06/2021', Output: Valid date.
Input: '31/12/2021', Output: Valid date.
Input: '01/01/2021', Output: Valid date.
Input: '31/11/2021', Output: Invalid date.
Input: '15/08/1947', Output: Valid date.
Input: '31/09/2021', Output: Invalid date.
Input: '00/01/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/00/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/01/2021', Output: Valid date.
Input: '31/11/2021', Output: Invalid date.
Input: '15/08/1947', Output: Valid date.
Input: '31/09/2021', Output: Invalid date.
Input: '00/01/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/00/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '15/08/1947', Output: Valid date.
Input: '31/09/2021', Output: Invalid date.
Input: '00/01/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/00/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '31/09/2021', Output: Invalid date.
Input: '00/01/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/00/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '00/01/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/00/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/00/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '32/01/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '01/13/2021', Output: Invalid date format. Please use DD/MM/YYYY.
Input: 'abc', Output: Invalid date format. Please use DD/MM/YYYY.
Input: '12-12-2021', Output: Invalid date format. Please use DD/MM/YYYY.
```